Recommender systems

Jelke Bloem

Text Mining
Amsterdam University College

Mar 31, 2026

Announcements

- Tuesday 07/04: Project topics and groups
- Friday 10/04: Programming Assignment 3 deadline
- Friday 10/04: Reading assignment test: BERT
- Tuesday 14/04: Project update 0 (repository+proposal)

Questions: Transformers

- Why did Vaswani et al. use wavelengths instead of simple number positional encoding to solve the order issue of the Transformers?
  - Relative position (+ absolute position)
  - Generalization to different sequence length
  - Easy to calculate dot product
  - Reminds of LSTM
  - Later: Rotary positional embeddings, variants
- Can the concept of self attention be applied to other types of models that are not necessarily neural networks, and can this improve their performance and interpretability as well?
- Was the transformer in the paper trained/tested mainly on translations because they require more attention to the syntactical aspect of language or is there another reason?

Questions: Transformers

- Are transformer models in other languages nowadays built based on English models or differently?
- Are there downsides to transformer models? E.g. any tasks or parts of language translation where a RNN is doing a better job?
  - Low-resource settings
  - Small devices
  - Streaming
  - Longer than transformer context window
  - Human agent simulation
- Are transformers now fully replacing previous models?

Overview

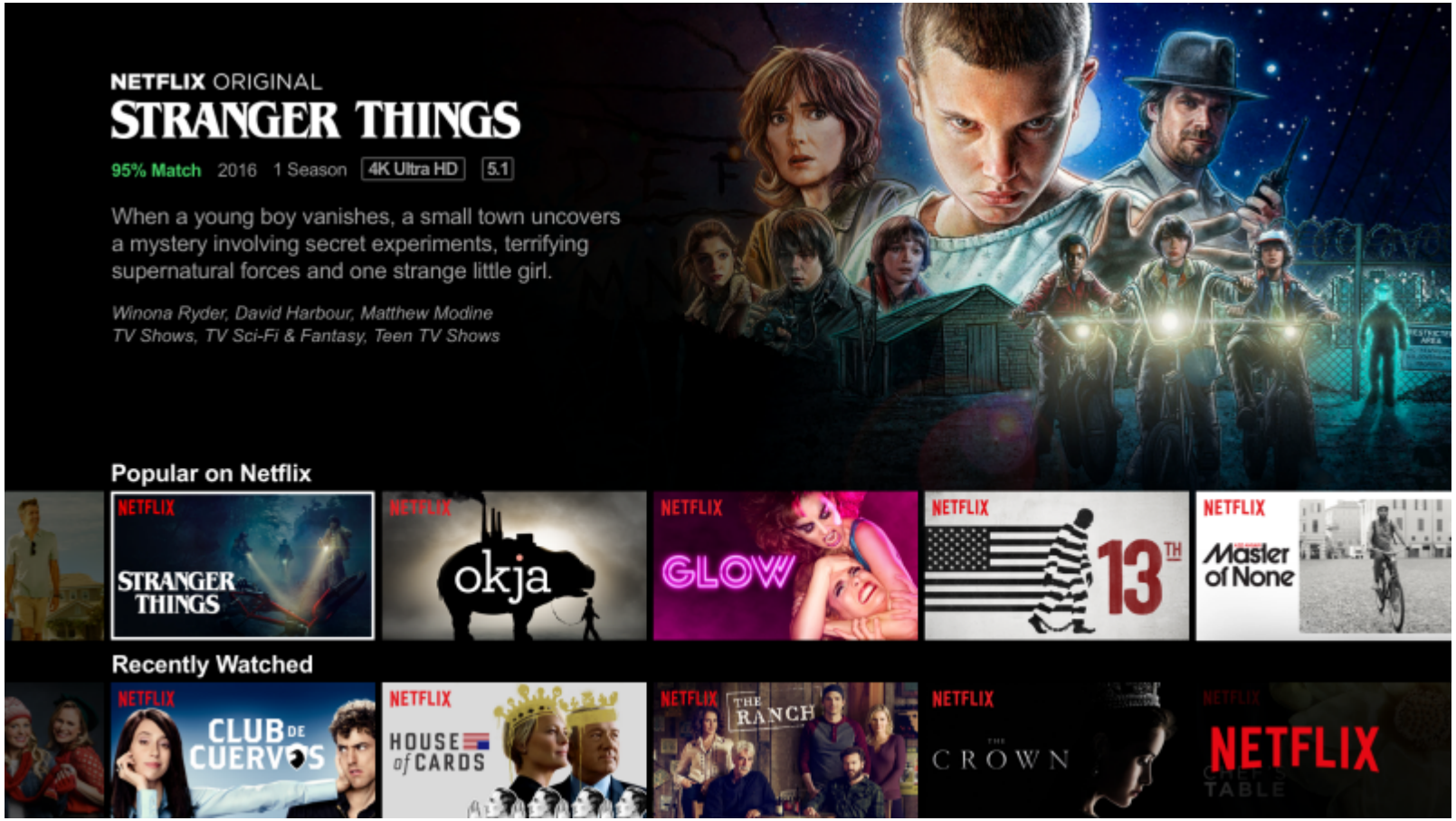
1. General concepts
2. k-Nearest Neighbours
3. More concepts
4. Evaluation

General concepts

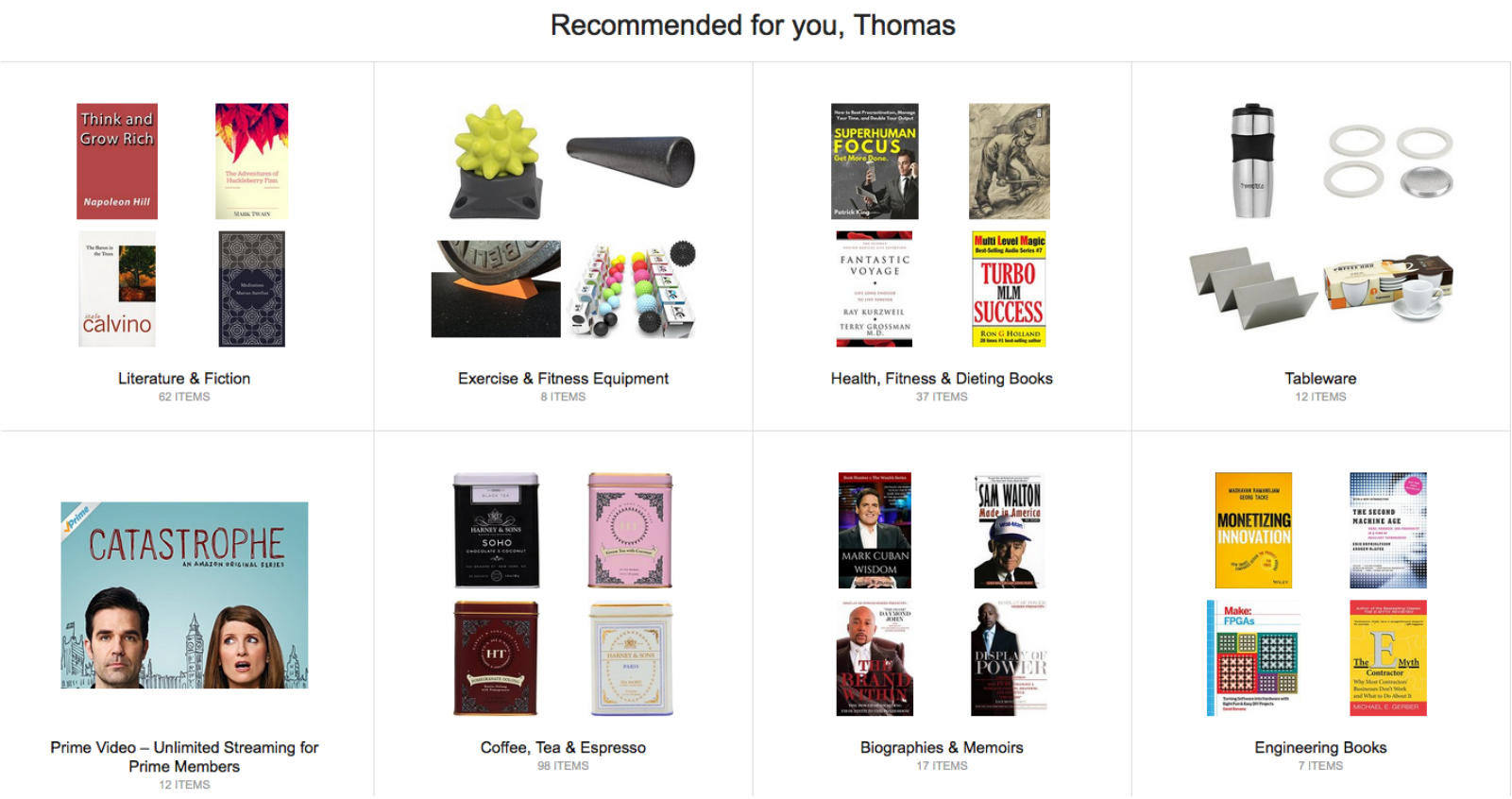
Definition

- Recommender systems (RS) focus on **predicting the relationship between users and items**.
- The relationship can take many forms: advertisement, preference (e.g., "likes"), retrieval (e.g., of information on the Web).
- How are preference data acquired?
  - **Implicitly**, e.g., videos watched.
  - **Explicitly**, e.g., likes on videos.
- How are users and items characterized?
  - Demographic features;
  - Social information;
  - Internet of Things (e.g., geolocation);
  - Contents (embedding features);
  - ...

Examples


NETFLIX ORIGINAL
STRANGER THINGS
95% Match 2016 1 Season 4K Ultra HD 5.1
When a young boy vanishes, a small town uncovers a mystery involving secret experiments, terrifying supernatural forces and one strange little girl.
Winona Ryder, David Harbour, Matthew Modine
TV Shows, TV Sci-Fi & Fantasy, Teen TV Shows
Popular on Netflix
NETFLIX
STRANGER THINGS
NETFLIX
okja
NETFLIX
GLOW
NETFLIX
13TH
NETFLIX
Master of None
Recently Watched
NETFLIX
CLUB DE CUERVOS
NETFLIX
HOUSE of CARDS
NETFLIX
THE RANCH
NETFLIX
THE CROWN
NETFLIX

Examples


Recommended for you, Thomas
Literature & Fiction
62 ITEMS
Exercise & Fitness Equipment
8 ITEMS
Health, Fitness & Dieting Books
37 ITEMS
Tableware
12 ITEMS
Prime Video – Unlimited Streaming for Prime Members
12 ITEMS
Coffee, Tea & Espresso
98 ITEMS
Biographies & Memoirs
17 ITEMS
Engineering Books
7 ITEMS

Examples

Search Videos
DISCOVER
WATCHLIST
RETURNING THE FAVOR
TODAY'S SPOTLIGHT
See All
LIVE
Angels at Indians
MLB Live by MLB
Clown Prank
Bae or Bail by A&E
MOST TALKED ABOUT
NAS DAILY
We Bought 1000 Burgers
Nas Daily
Raclette, New York City
The Great Cheese Hunt by Business Insider
WHAT FRIENDS ARE WATCHING
Latest Episodes
Saved Episodes
FOLLOWING
The Great Cheese Hunt
6 Episodes
Friday
Nas Daily
38 Episodes
Wednesday
The Showdown
4 Episodes
Tuesday
Safari Live
3 Episodes
Monday
Info
Comments
Up Next
Comeback Kids: Animal Edition
Following
S1 E4: Dog who lost her legs was determined to walk again
Linked Group
Rescue Pet Parents
Like
Write a comment...
Share

Examples

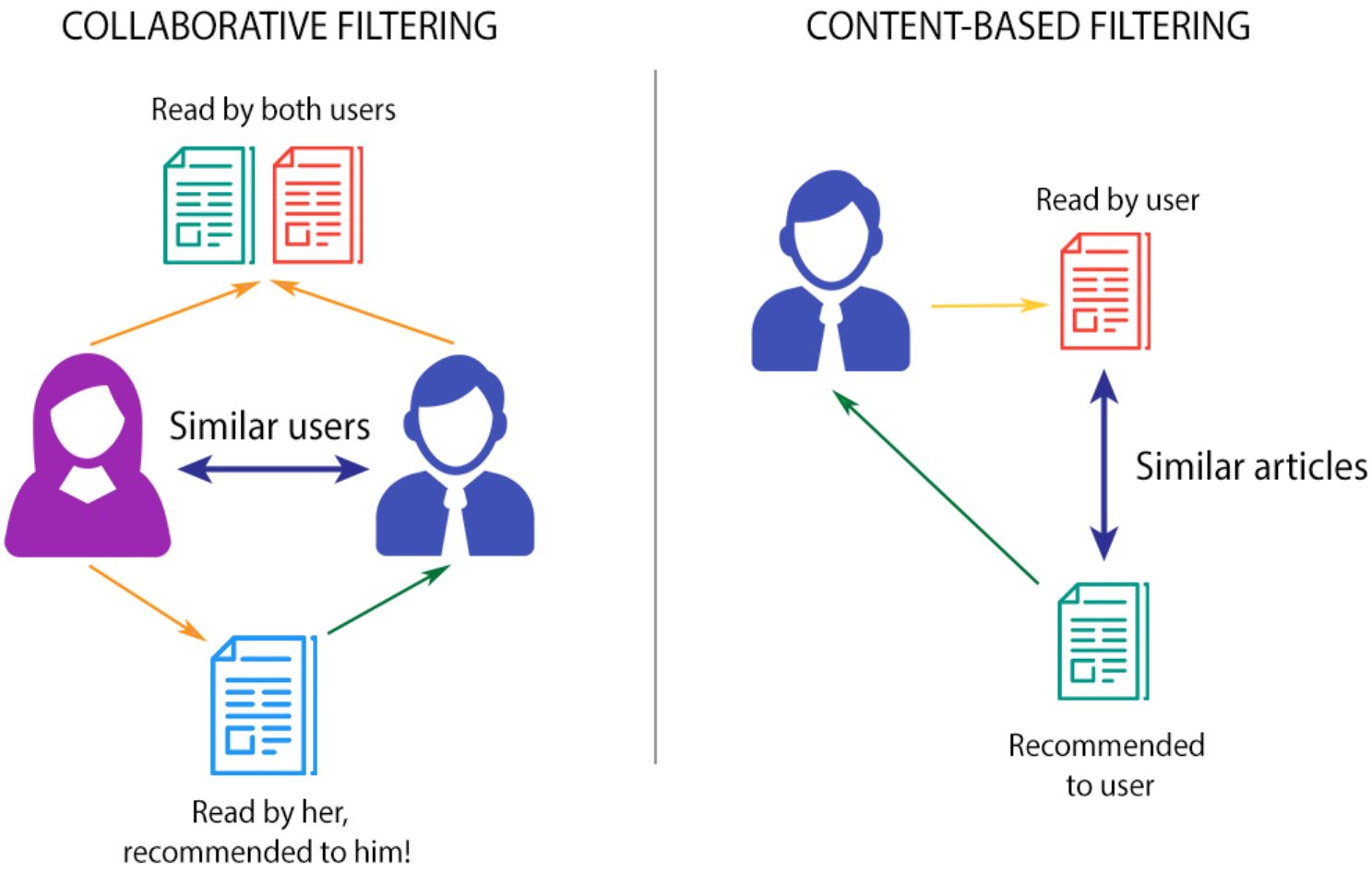
Notebook 9: MovieLens 1M dataset

- 1 million ratings by 6040 users
- 3592 movies
- User gender, age range, occupation, zip-code
- Movie genre

Classification of RS

- **Approach: collaborative vs content-based filtering.**
- **Methods: memory vs model-based.**
- Algorithms and techniques.
- Kind and quality of recommendation.
- ...

Collaborative vs content-based filtering



Collaborative vs content-based filtering

- **Collaborative filtering (CF)**: for a given user, recommendations are based on the preferences from those users that have most in common with him/her.
- Intuition: people with similar taste will make similar choices.
- **Content-based filtering (CBF)**: recommend items similar to those that the user considered positively in the past.
- Intuition: recommendations are based on the relevant characteristics of the objects intended for recommendation (text, video, sound).

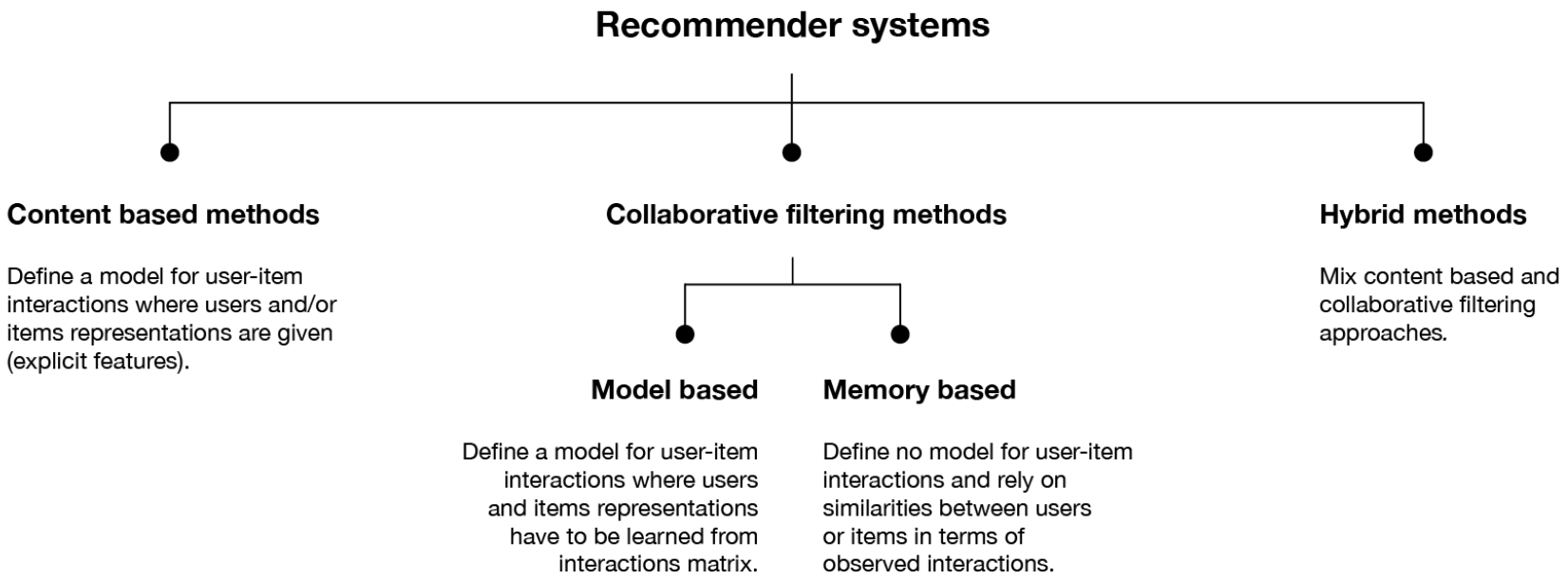
Collaborative vs content-based filtering

- Collaborative filtering (CF): popular methods include **k-Nearest Neighbours** and **matrix factorization**.
- Content-based filtering (CBF): given user profiles and the properties of items, standard algorithms are used to decide if an item should be recommended to a given user (**classification**), or how the given user would score a certain item (**regression**).
- Note: it is rare to find "pure" filtering approached: most systems use combinations, they are thus **hybrid**, and use all the state-of-the-art machinery available (neural networks).

Methods

- **Memory-based**: based on the direct use of data (i.e., they use only the matrix of user-items ratings). These methods are also called *non-parametric* in ML parlance. Example: k-NN.
- **Model-based**: based on a model which uses the data in turn. These methods are also called *parametric* in ML parlance. Example: matrix factorization.
- *Reminder: A parametric model is one that can be parametrized by a finite number of parameters.* We encountered this before.

Classification of RS



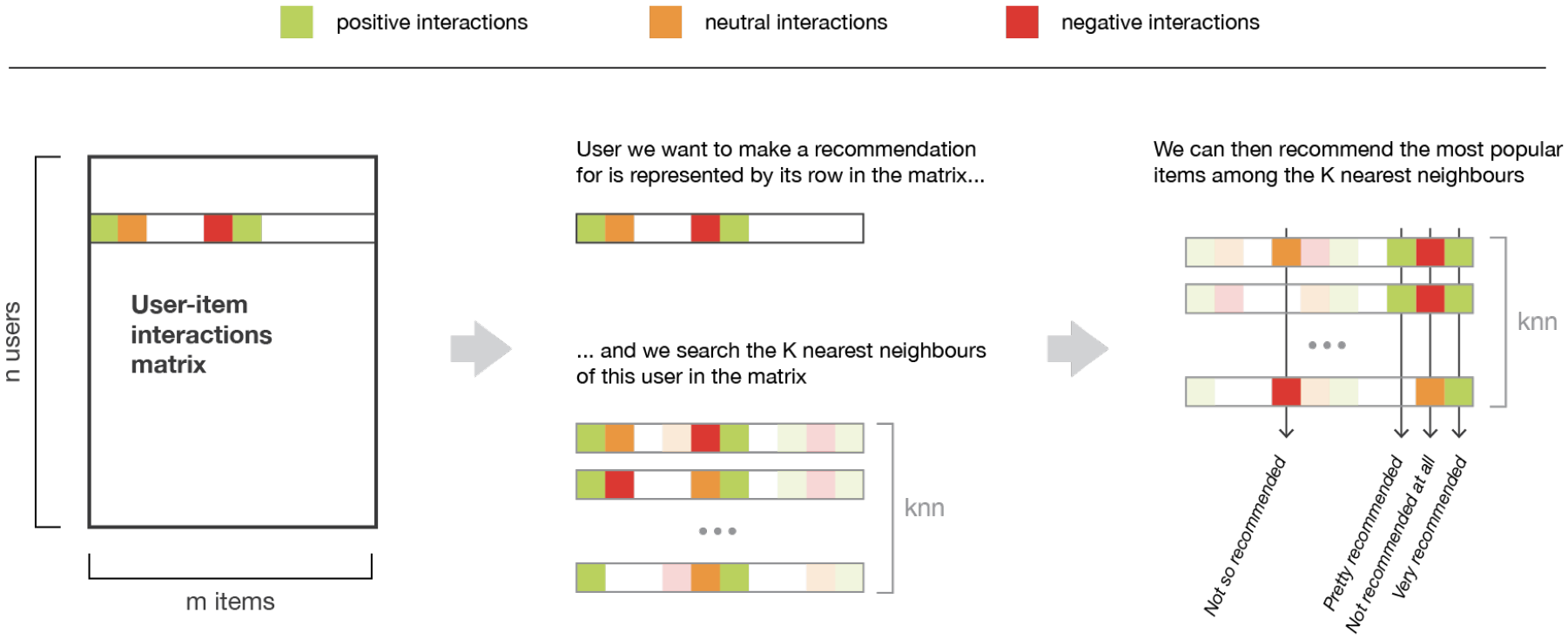
https://towardsdatascience.com/introduction-to-recommender-systems-6c66cf15ada

k-Nearest Neighbours

k-Nearest Neighbours

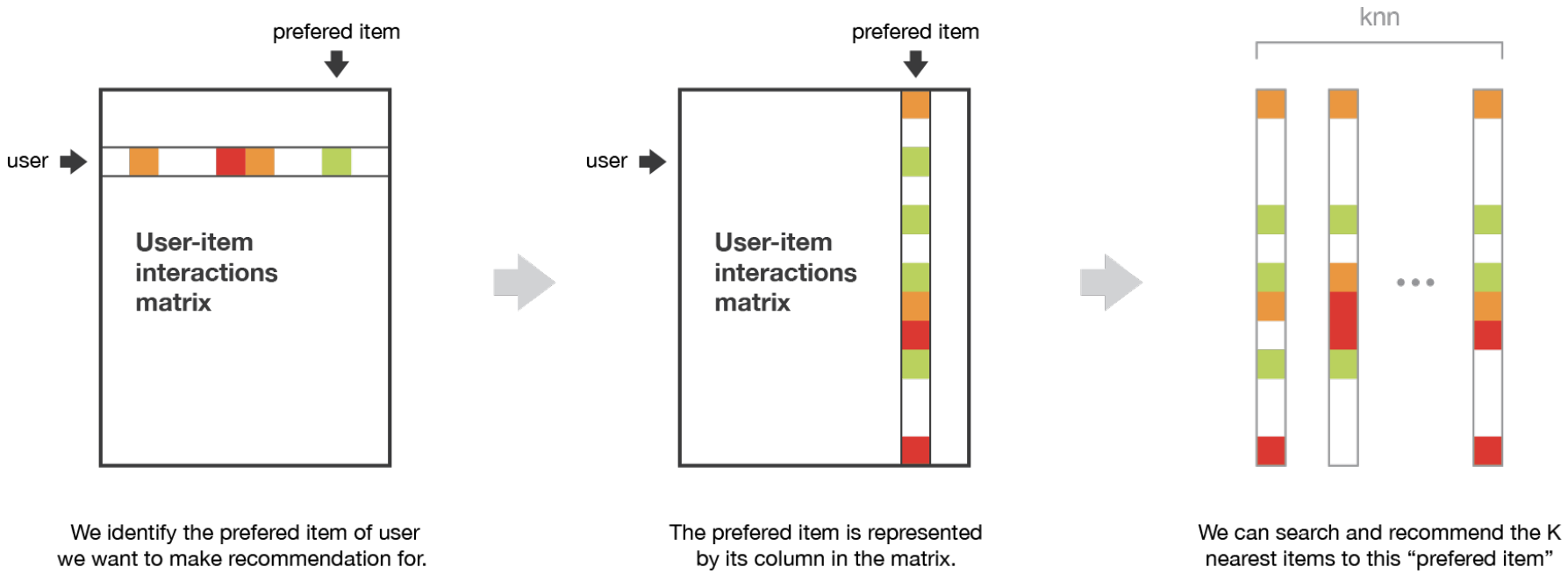
- The most popular *memory-based* algorithm for *collaborative filtering* recommendations;
- the method most popular at the beginning of RS;
- conceptually simple and easy to implement, and it generally produces good predictions;
- based on similarity measures;
- two main versions: **user-to-user** and **item-to-item**.
- Remember the word-context matrix? Let us build a **user-item matrix** now!

User-to-user



https://towardsdatascience.com/introduction-to-recommender-systems-6c66cf15ada

Item-to-item



https://towardsdatascience.com/introduction-to-recommender-systems-6c66cf15ada

k-Nearest Neighbours with user-to-user

Given an active user a:

1. use a similarity measure to determine the k most-similar users to a;
2. obtain the prediction on item i for user a by using one aggregation approaches on the item ratings in a's neighborhood:
   - average
   - weighted sum
   - adjusted weighted aggregation (deviation-from-mean)
   - ...
3. choose the top-n items by selecting the n items with the highest scores calculated by applying the previous steps on the items that have not yet been rated by user a.

Notebook 9: User-to-user kNN example with these steps

k-Nearest Neighbours with item-to-item

Given an active user a:

1. use a similarity measure to determine the k most-similar items to those already rated by a;
2. choose the top-n items by selecting the n items with the highest scores on the items that have not yet been rated by user a.

Notebook 9: Item-to-item kNN example with these steps (end of notebook)

Similarity measures

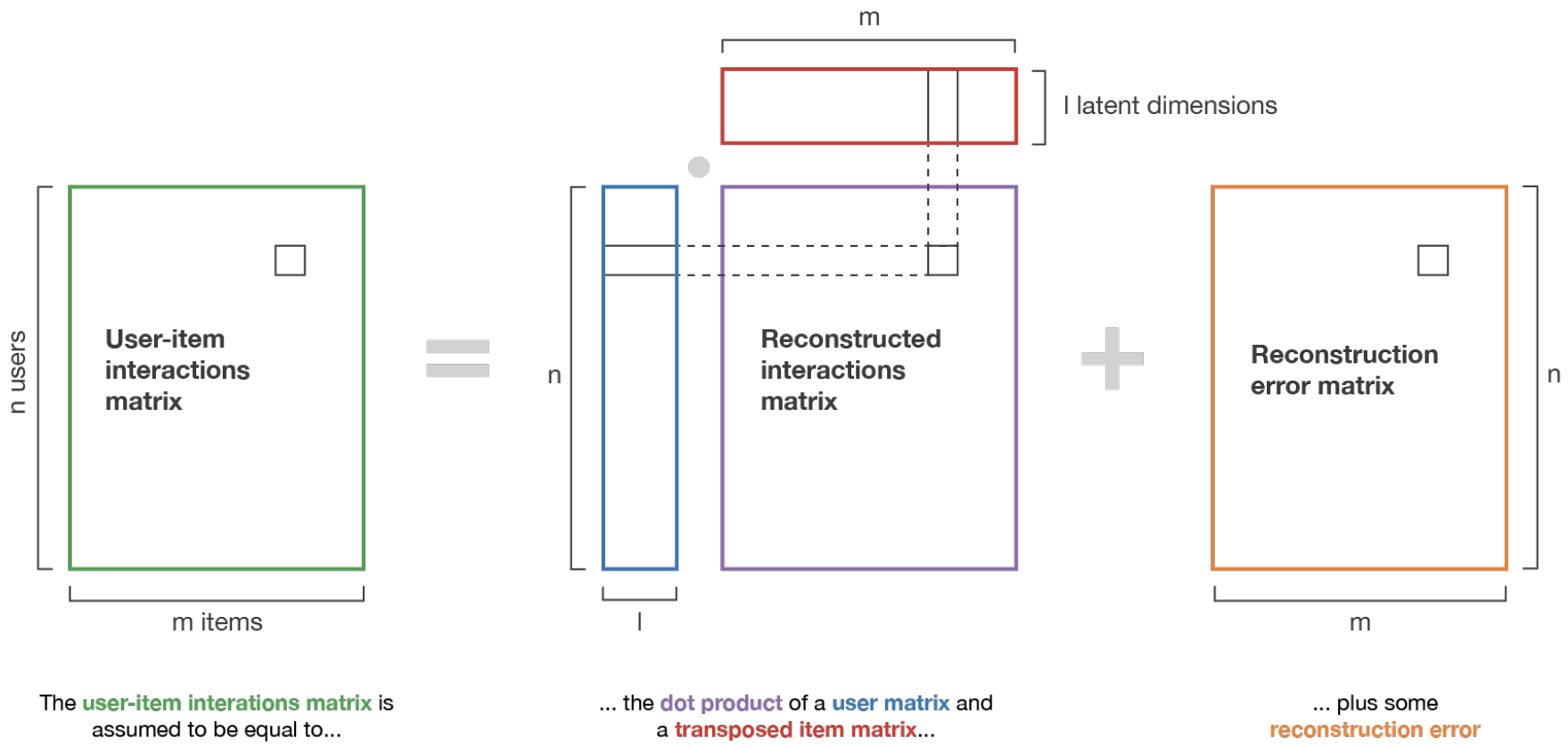
1. Many options, recall class on word vectors and see `https://docs.scipy.org/doc/scipy/reference/spatial.distance.html`.
2. *Euclidean*.
3. *Cosine*: considers the angle between two vectors, not the magnitude (norm).
4. *Pearson* correlation: strength of linear dependence.
5. It is usually easy to *turn a distance into a similarity* (take the inverse or subtract from the max value it can take).
6. More during the lab.. Notebook 9, "Similarity measures" (and how to choose)

User-to-user vs item-to-item

1. User-to-user does not **scale** well: $O(ndk)$ with n users, d items and k neighbours. Imagine how many users and items are on Amazon..
2. Item-to-item k-NN usually reduces this scalability problem. $O(d^2)$ but in practice often much less complex, *why?*
3. In practice, *approximate k-NN* methods as well as *caching* help a lot.
4. Sparsity is a huge issues for both approaches.
5. User-to-user k-NN is simple to implement and often more appropriate with smaller in-memory datasets that change frequently.
6. The two approaches can be combined.

More concepts

Matrix factorization



https://towardsdatascience.com/introduction-to-recommender-systems-6c66cf15ada

Matrix factorization

1. We can solve the sparsity problem and use **dense vectors**, as we did with word vectors.
2. We can then use k-NN on dense vectors instead (or as features for other models).
3. Matrix factorization, e.g., SVD, is done using standard optimization techniques: define a loss function (e.g., MSE of the original vs reconstructed matrix) and minimize it.

Cold-start problem

1. Issue raising when we do not have enough data to make reliable recommendations:
   - **new community or user**: when starting up a new RS or for a new user. Solution 1: encourage users to make ratings (e.g., Netflix). Solution 2: use CF recommendations only when you have enough data. Solution 3: other data and hybrid.
   - **New item**: new items usually are not rated, so that they are not recommended. Not a big deal if you have items that can be discovered also by other means. Solution: to have a group of motivated users that rate new items or use hybrid.

More challenges

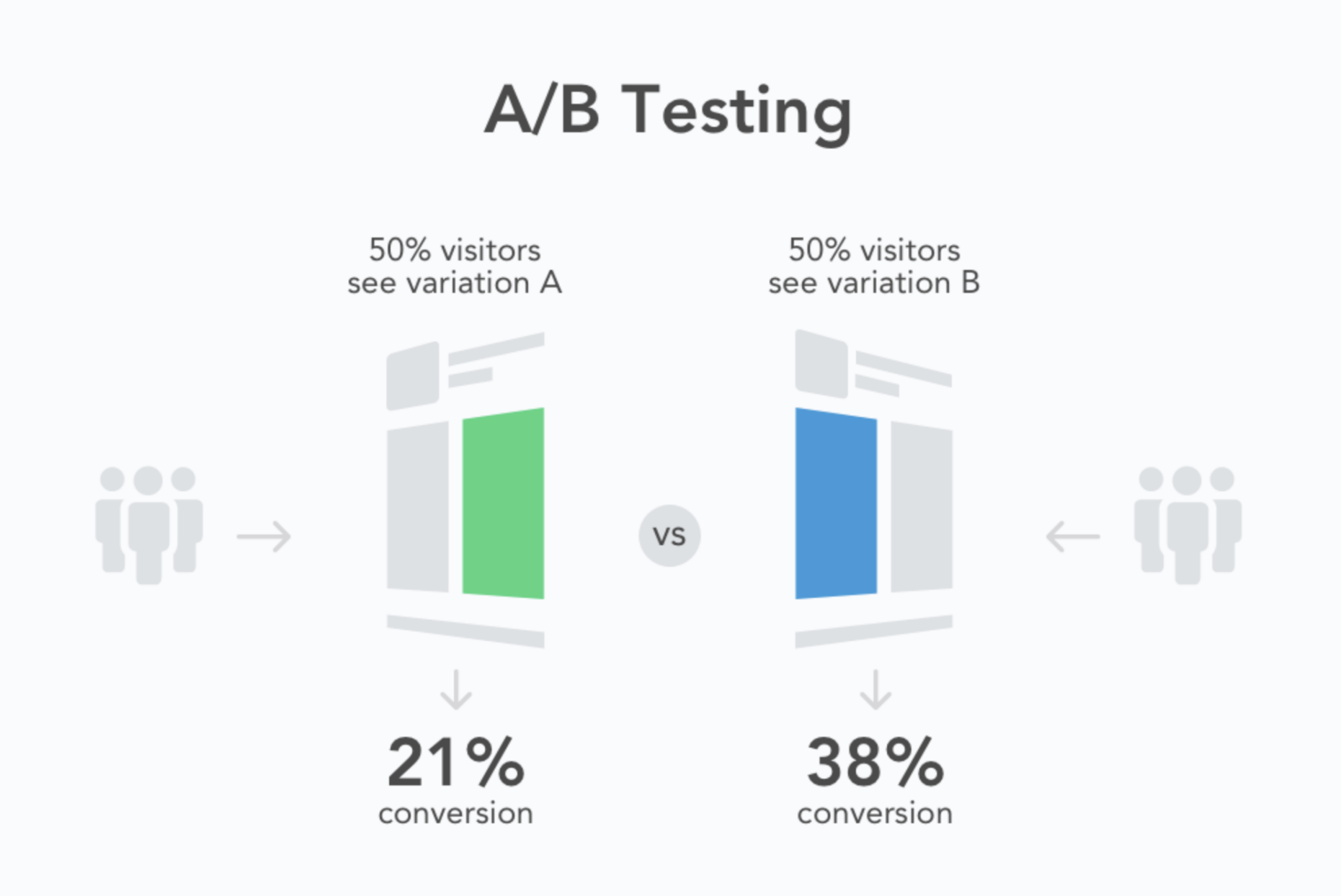
1. **Rich get richer**: when highly popular items dominate your RS.
2. Information confinement area (or "**filter bubbles**"): when serendipity does not happen and users get too much of the same.
3. Solutions: mix signals, emphasize recency.

Evaluation

Metrics

1. **Prediction metrics** between predicted and actual: mean absolute/squared error, root mean squared error, normalized mean absolute error, ..
2. **Set recommendation metrics**: evaluate the reduced set of recommendations returned by the RS using standard metrics (precision, recall, F-measure, accuracy).
3. **Rank recommendation metrics**: take into consideration the ranking (ordering) of the results.
4. **Novelty metrics**: degree of difference between the recommended items and those known by the user.
5. **Diversity metrics**: degree of differentiation among the recommended items (to keep variety high).
6. **Stability metrics**: how much the predictions produced by an RS change in a given period of time.

Users


A/B Testing
50% visitors
see variation A
50% visitors
see variation B
VS
21%
conversion
38%
conversion

References

1. J. Bobadilla, F. Ortega, A. Hernando and A. Gutiérrez (2013): *Recommender systems survey*.
2. T. Segaran (2007): *Programming Collective Intelligence*. (Chapters 2 and 3).
3. Recent neural network-based approaches:
   - `https://link.springer.com/article/10.1007/s10791-017-9321-y`
   - `https://arxiv.org/pdf/1707.07435.pdf`
   - `https://arxiv.org/pdf/1907.06902.pdf`